

Claude.ai - HW2 should be a 7×6 grid, not 6×7 like in the text
can change dimensions per l(a)
we need to use CS4300 principles, like SOFA and keep it simple stupid
the professor will tell us which parking spot to park in during the
demo. PDF for this assignment is in the homework folder

Atari Games

Played by
Deep Q Learning
Mnih et al., 2013

Atari Pong

- **Pong** is a 1972 sports video game developed and published by Atari, Inc., for arcades.
- *Pong* was the first commercially successful video game, helping to establish the video game industry.
- Pong is a two-dimensional sports game that simulates table tennis.
- The player controls a paddle by moving it vertically across the left or right side of the screen.
- They can compete against another player controlling a second paddle on the opposing side. Players use the paddles to hit a ball back and forth.
- The goal is for each player to reach eleven points before the opponent; points are earned when one fails to return the ball to the other.
- <https://www.youtube.com/watch?v=fiShX2pTz9A>

Atari Breakout

- **Breakout** is a 1976 action video game developed and published by Atari, Inc. for arcades.
- Breakout begins with eight rows of bricks, with two rows each of a different color.
- Using a single ball, the player must knock down as many bricks as possible by using the walls and the paddle below to hit the ball against the bricks and eliminate them.
- https://www.youtube.com/watch?v=sxzGhG_38pc

Atari Space Invaders

- **Space Invaders** is a 1980 fixed shooter video game developed and published by Atari, Inc. for the Atari 2600 as a port of Taito's 1978 arcade game of the same name.
- Space Invaders has the player at war with enemies from outer space.
- The player uses the joystick to move left and right and presses the red button to fire a laser cannon.
- The goal is to earn as many points as possible by destroying the enemies with a laser cannon and to eliminate them before they reach the bottom of the screen.
- <https://www.youtube.com/watch?v=opru6qPsPa4>

Atari Seaquest

- **Seaquest** is an Atari 2600 video game published by Activision in 1983.
- The game is an underwater shooter in which the player controls a submarine on a mission to rescue divers.
- The player uses a submarine to shoot at enemies and rescue divers. Hazards include sharks which attack divers and the player and enemy submarines, which can shoot torpedoes at the player's submarine.
- The player must ward off the enemies by firing an unlimited supply of torpedoes while trying to rescue divers swimming through the water.
- <https://www.youtube.com/watch?v=lr1t57ltCpw&t=236s>

Atari Beam Rider

- **Beamrider** is a 1983 fixed shooter video game published by Intellivision. The game was ported to the Atari 2600 (with a slightly reduced feature set).
- Beamrider takes place above Earth's atmosphere, where a large alien shield called the Restrictor Shield surrounds the Earth.
- The player's objective is to clear the Shield's 99 sectors of alien craft while piloting the Beamrider ship.
- The Beamrider is equipped with a short-range laser lariat and a limited supply of torpedoes. The player is given three at the start of each sector.
- <https://www.youtube.com/watch?v=dgTxca0pdac>

Tabular Q-Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$
 Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
 Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose A from S using policy derived from Q (e.g., ϵ -greedy) *get*
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ *learn*
 $S \leftarrow S'$
 until S is terminal *learning by the agent happens once*

this looks familiar

access Q-table to obtain all actions in a state. Find best action. Give highest prob of being chosen, give a little chance to all actions

The Q table is used in two places:

To choose an action: Choose A from S using policy derived from Q (e.g., ϵ -greedy)

To update Q table ie., learn new values:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

want to replace Q-table with a NN

need a Q-neural network

given a state — get all actions available in that state
 frame of a game
 $q(s, a)$ values, regression learning
 — get max $q(s, a)$ value, probabilities to each of the actions
 — take action a and observe R, S'
 — feed state S' into NN again but this time with loss function

Deep Q-learning

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
 Initialize action-value function Q with random weights
for episode = 1, M **do**
 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
 for $t = 1, T$ **do**
 With probability ϵ select a random action a_t
 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
 Execute action a_t in emulator and observe reward r_t and image x_{t+1}
 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
 end for
end for

learning is decoupled from what it is doing right now.

```
        Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3  
    end for  
end for
```

The Q-table is replaced by a neural network in the same two places